Linearized Longitudinal Flight Dynamics

Derive, linearize, deploy, and reuse a longitudinal aircraft model from first principles. The goal is not to replace high-fidelity flight dynamics tools; it is to show where symbolic analysis adds engineering value before those tools are used.

Workflow: aerodynamic coefficients to wind-axis equations of motion to exact cruise trim to symbolic A/B Jacobians to modal analysis to stability-derivative dominance to transonic Mach-drag and Mach-moment extension to generated MATLAB® functions to uncertainty propagation

Requires: Symbolic Math Toolbox™, Aerospace Toolbox

Copyright 2026 The MathWorks, Inc.

1. Longitudinal Aircraft Equations of Motion

Start with a compact 3DOF longitudinal model. The states are airspeed V, angle of attack α, pitch rate q, and pitch attitude θ. The controls are elevator deflection δ_e and body-axis thrust T.

```
syms V alpha q theta real
syms de thrust real
syms CL0 CLa CLq CLde real
syms CD0 K_drag real
syms Cm0_ac Cma_ac Cmq Cmde real
syms h_cg real
syms S cbar mass Iyy real
syms rho g real
states = [V, alpha, q, theta];
controls = [de, thrust];
```

Aerodynamic Coefficients

The lift model includes static angle-of-attack, pitch-rate, and elevator terms. Drag uses a parabolic polar. The pitching moment is split into an aerodynamic-center contribution plus the moment from lift acting through the CG offset.

```
CL = CL0 + CLa*alpha + CLq*(cbar/(2*V))*q + CLde*de;
CD = CD0 + K_drag*CL^2;
Cm = Cm0_ac + Cma_ac*alpha + CL*h_cg + Cmq*(cbar/(2*V))*q + Cmde*de;
qbar = rho*V^2/2;
```

```
L_force = qbar*S*CL;
D_force = qbar*S*CD;
M_moment = qbar*S*cbar*Cm;
```

Translational dynamics are written in wind axes because V and α are wind-axis states. With flight-path angle $\gamma = \theta - \alpha$, the axial and normal wind-axis force balances are used directly.

```
gamma = theta - alpha;
Fx_wind = thrust*cos(alpha) - D_force - mass*g*sin(gamma);
Fz_wind = -thrust*sin(alpha) - L_force + mass*g*cos(gamma);
Vdot = Fx_wind/mass;
alphadot = Fz_wind/(mass*V) + q;
qdot = M_moment/Iyy;
thetadot = q;
```

Nonlinear EOM:

```
f = [Vdot; alphadot; qdot; thetadot]
```

```
f =
```

$$\begin{bmatrix} \dfrac{\text{thrust}\cos(\alpha) + g\,\text{mass}\sin(\alpha-\theta) - \dfrac{S\,V^2\,\rho\left(\text{CD}_0 + K_{\text{drag}}\left(\text{CL}_0 + \text{CLa}\,\alpha + \text{CLde}\,\text{de} + \dfrac{\text{CLq}\,\text{cbar}\,q}{2\,V}\right)^2\right)}{2}}{\text{mass}} \\ q - \dfrac{\text{thrust}\sin(\alpha) - g\,\text{mass}\cos(\alpha-\theta) + \dfrac{S\,V^2\,\rho\left(\text{CL}_0 + \text{CLa}\,\alpha + \text{CLde}\,\text{de} + \dfrac{\text{CLq}\,\text{cbar}\,q}{2\,V}\right)}{2}}{V\,\text{mass}} \\ \dfrac{S\,V^2\,\text{cbar}\,\rho\left(\text{Cm}_{0,\text{ac}} + \text{Cma}_{\text{ac}}\,\alpha + \text{Cmde}\,\text{de} + h_{\text{cg}}\left(\text{CL}_0 + \text{CLa}\,\alpha + \text{CLde}\,\text{de} + \dfrac{\text{CLq}\,\text{cbar}\,q}{2\,V}\right) + \dfrac{\text{Cmq}\,\text{cbar}\,q}{2\,V}\right)}{2\,\text{Iyy}} \\ q \end{bmatrix}$$

2. Cruise Trim and Linearization

Use a representative narrow-body transport at Mach 0.78 and FL350. `atmosisa` supplies density and speed of sound so the cruise dynamic pressure is tied to a physical atmosphere.

```
alt_m = 10668;
[T_atm, a_sound, ~, rho_val] = atmosisa(alt_m);
M_cruise = 0.78;
V0 = M_cruise*a_sound;
S_val = 125;
cbar_val = 4.2;
mass_val = 60000;
Iyy_val = 3.5e6;
g_val = 9.80665;
CL0_val = 0.10;
CLa_val = 5.5;
CLq_val = 7.0;
CLde_val = 0.36;
CD0_val = 0.018;
K_val = 0.045;
Cm0ac_val = 0.04;
Cmaac_val = -0.3;
h_cg_val = -0.15;
Cmq_val = -18.0;
Cmde_val = -1.4;
params = [rho, S, cbar, mass, Iyy, g, ...
    CL0, CLa, CLq, CLde, CD0, K_drag, ...
    Cm0_ac, Cma_ac, h_cg, Cmq, Cmde];
vals = [rho_val, S_val, cbar_val, mass_val, Iyy_val, g_val, ...
    CL0_val, CLa_val, CLq_val, CLde_val, CD0_val, K_val, ...
    Cm0ac_val, Cmaac_val, h_cg_val, Cmq_val, Cmde_val];
disp(join([
    compose("Altitude: %d m (FL350)", alt_m)
    compose("Density: %.4f kg/m^3", rho_val)
    compose("Speed of sound: %.1f m/s", a_sound)
    compose("Cruise true airspeed: %.1f m/s", V0)
], newline))
```

```
Altitude: 10668 m (FL350)
Density: 0.3796 kg/m^3
Speed of sound: 296.5 m/s
Cruise true airspeed: 231.3 m/s
```

Exact Steady-Level Trim

At steady level flight, $q_0 = 0$, $\theta_0 = \alpha_0$, and all state derivatives are zero. We eliminate thrust with axial wind-axis balance, then solve normal-force and pitch-moment balance for α_0 and δ_{e0}.

```
qbar0 = 0.5*rho_val*V0^2;
alpha_guess = (mass_val*g_val/(qbar0*S_val) - CL0_val)/CLa_val;
CL_guess = CL0_val + CLa_val*alpha_guess;
de_guess = -(Cm0ac_val + Cmaac_val*alpha_guess + CL_guess*h_cg_val)/(Cmde_val + CLde_val*h_cg_val);
thrustTrimExpr = (D_force + mass*g*sin(gamma))/cos(alpha);
trim_eqs = subs([subs(Fz_wind, thrust, thrustTrimExpr) == 0; M_moment == 0], ...
    [params, V, q, theta], ...
    [vals, V0, 0, alpha]);
trim_sol = vpasolve(trim_eqs, [alpha, de], [alpha_guess, de_guess]);
alpha0 = double(trim_sol.alpha);
de0 = double(trim_sol.de);
theta0 = alpha0;
thrust0 = double(subs(thrustTrimExpr, ...
    [params, V, alpha, q, theta, de], ...
    [vals, V0, alpha0, 0, theta0, de0]));
trimResidual = double(subs(f, [params, states, controls], ...
    [vals, V0, alpha0, 0, theta0, de0, thrust0]));
disp(join([
    compose("Trim AoA: %.2f deg", rad2deg(alpha0))
    compose("Trim elevator: %.2f deg", rad2deg(de0))
    compose("Trim thrust: %.0f N", thrust0)
    compose("Trim residual norm: %.2e", norm(trimResidual))
], newline))
```

```
Trim AoA: 3.90 deg
Trim elevator: -2.03 deg
Trim thrust: 35103 N
Trim residual norm: 4.54e-17
```

Symbolic A and B Matrices

The symbolic Jacobians preserve the formula behind each stability derivative. The same derivation gives the control-input matrix, so the result can move directly into control-design workflows.

```
A_sym = simplify(jacobian(f, states));
B_sym = simplify(jacobian(f, controls));
```

Linearized state matrix at cruise trim:

```
A_Val = double(subs(A_sym, [params, states, controls], ...
    [vals, V0, alpha0, 0, theta0, de0, thrust0]))
```

```
A_Val = 4×4
   -0.0050    4.9323   -0.0559   -9.8066
   -0.0004   -0.5055    0.9942         0
    0.0000   -1.7135   -0.2634         0
         0         0    1.0000         0
```

Input matrix columns are elevator deflection and thrust:

```
B_Val = double(subs(B_sym, [params, states, controls], ...
    [vals, V0, alpha0, 0, theta0, de0, thrust0]))
```

```
B_Val = 4×2
   -0.3164    0.0000
   -0.0329   -0.0000
   -2.2146         0
         0         0
```

Modal Analysis

The eigenvalues expose the short-period and phugoid modes. Because this is a deliberately compact teaching model, the modal values are illustrative rather than certification-quality.

```
eigvals = eig(A_Val);
wn = abs(eigvals);
zeta = -real(eigvals)./wn;
periods = 2*pi./abs(imag(eigvals));
[~, idx] = sort(wn, "descend");
```

```
sp_idx = idx(1:2);
ph_idx = idx(3:4);
fprintf(['=== Subsonic Modal Summary ===\n' ...
    'Short-period: wn = %.3f rad/s, zeta = %.3f\n' ...
    'Phugoid:      wn = %.4f rad/s, zeta = %.4f, period = %.0f s\n'], ...
    wn(sp_idx(1)), zeta(sp_idx(1)), ...
    wn(ph_idx(1)), zeta(ph_idx(1)), periods(ph_idx(1)))
```

```
=== Subsonic Modal Summary ===
Short-period: wn = 1.355 rad/s, zeta = 0.284
Phugoid:      wn = 0.0578 rad/s, zeta = 0.0339, period = 109 s
```

3. Stability-Derivative Dominance

This is where symbolic analysis pays off: a numeric linearization gives a number, while a symbolic Jacobian gives the formula that created the number.

Pitch stiffness derivative M_α:

```
Malpha_sym = A_sym(3, 2)
```

```
Malpha_sym =
```

$$\frac{S\,V^2\,\mathrm{cbar}\,\rho\,(\mathrm{Cma_{ac}} + \mathrm{CLa}\,h_{\mathrm{cg}})}{2\,\mathrm{Iyy}}$$

```
Malpha_terms = children(expand(Malpha_sym));
if isa(Malpha_terms, "cell")
    Malpha_terms = [Malpha_terms{:}];
end
Malpha_total = double(subs(Malpha_sym, [params, states, controls], ...
    [vals, V0, alpha0, 0, theta0, de0, thrust0]));
fprintf('=== M_alpha Term Breakdown ===\n')
```

```
=== M_alpha Term Breakdown ===
```

```
for k = 1:numel(Malpha_terms)
    termVal = double(subs(Malpha_terms(k), [params, states, controls], ...
        [vals, V0, alpha0, 0, theta0, de0, thrust0]));
    pct = 100*termVal/Malpha_total;
```

```
    fprintf('  Term %d: %+.4f /s^2  (%+.1f%%)\n    %s\n', k, termVal, pct, char(Malpha_terms(k)))
end
```

```
  Term 1: -0.4569 /s^2  (+26.7%)
    (Cma_ac*S*V^2*cbar*rho)/(2*Iyy)
  Term 2: -1.2566 /s^2  (+73.3%)
    (CLa*S*V^2*cbar*h_cg*rho)/(2*Iyy)
```

```
fprintf('  Total M_alpha = %.4f /s^2\n', Malpha_total)
```

```
  Total M_alpha = -1.7135 /s^2
```

Exact Sensitivities

Differentiate the symbolic derivative with respect to the underlying aerodynamic data. This tells a stability engineer which wind-tunnel measurements and CG tolerances deserve the most attention.

```
sens_coeffs = [Cma_ac, CLa, h_cg, Cmq, CLq];
sens_names = ["Cma_ac", "CLa", "h_cg", "Cmq", "CLq"];
fprintf('=== Sensitivity of M_alpha ===\n')
```

```
=== Sensitivity of M_alpha ===
```

```
for k = 1:numel(sens_coeffs)
    dMalpha_dC = diff(Malpha_sym, sens_coeffs(k));
    dMalpha_dC_Val = double(subs(dMalpha_dC, [params, states, controls], ...
        [vals, V0, alpha0, 0, theta0, de0, thrust0]));
    fprintf('  dM_alpha/d%s = %+.4f /s^2 per unit\n', sens_names(k), dMalpha_dC_Val)
end
```

```
  dM_alpha/dCma_ac = +1.5231 /s^2 per unit
  dM_alpha/dCLa = -0.2285 /s^2 per unit
  dM_alpha/dh_cg = +8.3770 /s^2 per unit
  dM_alpha/dCmq = +0.0000 /s^2 per unit
  dM_alpha/dCLq = +0.0000 /s^2 per unit
```

```
Zalpha_sym = A_sym(2, 2);
Zalpha_Val = double(subs(Zalpha_sym, [params, states, controls], ...
    [vals, V0, alpha0, 0, theta0, de0, thrust0]));
dZalpha_dCLa = diff(Zalpha_sym, CLa);
dZalpha_dCLa_Val = double(subs(dZalpha_dCLa, [params, states, controls], ...
```

```
    [vals, V0, alpha0, 0, theta0, de0, thrust0]));
fprintf(['\nZ_alpha = %+.4f /s\n' ...
    'dZ_alpha/dCLa = %+.4f /s per unit\n'], ...
    Zalpha_Val, dZalpha_dCLa_Val)
```

```
Z_alpha = -0.5055 /s
dZ_alpha/dCLa = -0.0915 /s per unit
```

4. Transonic Regime Extension

The subsonic model has no explicit Mach dependence, so M_u is essentially zero at trim. Near drag divergence, that is incomplete: wave drag affects axial damping, and aft aerodynamic-center shift creates Mach-tuck pitching moment.

```
Xu_sym = A_sym(1, 1);
Mu_sym = A_sym(3, 1);
Xu_Val = double(subs(Xu_sym, [params, states, controls], ...
    [vals, V0, alpha0, 0, theta0, de0, thrust0]));
Mu_Val = double(subs(Mu_sym, [params, states, controls], ...
    [vals, V0, alpha0, 0, theta0, de0, thrust0]));
fprintf(['=== Subsonic Speed Derivatives ===\n' ...
    'X_u = %+.6f /s\n' ...
    'M_u = %+.6f /s^2\n'], Xu_Val, Mu_Val)
```

```
=== Subsonic Speed Derivatives ===
X_u = -0.005047 /s
M_u = +0.000000 /s^2
```

Piecewise Drag and Pitching-Moment Model

Add a cubic wave-drag ramp and a quadratic Mach-dependent pitching-moment increment above drag-divergence Mach number M_{dd}. The coefficients are conceptual-design placeholders, not aircraft-specific data.

```
syms M_var Mdd Kwave a_sym positive
syms CmMach real
CD_sub = CD0 + K_drag*CL^2;
dCD_wave = Kwave*(M_var - Mdd)^3;
CD_trans = piecewise(M_var < Mdd, CD_sub, M_var >= Mdd, CD_sub + dCD_wave);
dCm_mach = CmMach*(M_var - Mdd)^2;
```

```
Cm_trans = piecewise(M_var < Mdd, Cm, M_var >= Mdd, Cm + dCm_mach);
M_expr = V/a_sym;
CD_transonic = subs(CD_trans, M_var, M_expr);
Cm_transonic = subs(Cm_trans, M_var, M_expr);
```

Transonic drag coefficient:

```
CD_trans
```

```
CD_trans =
```

$$\begin{cases} \mathrm{CD}_0 + K_{\mathrm{drag}} \left(\mathrm{CL}_0 + \mathrm{CLa}\,\alpha + \mathrm{CLde}\,\mathrm{de} + \dfrac{\mathrm{CLq}\,\mathrm{cbar}\,q}{2\,V}\right)^2 & \text{if } M_{\mathrm{var}} < \mathrm{Mdd} \\ \mathrm{CD}_0 + \mathrm{Kwave}\ (M_{\mathrm{var}} - \mathrm{Mdd})^3 + K_{\mathrm{drag}} \left(\mathrm{CL}_0 + \mathrm{CLa}\,\alpha + \mathrm{CLde}\,\mathrm{de} + \dfrac{\mathrm{CLq}\,\mathrm{cbar}\,q}{2\,V}\right)^2 & \text{if } \mathrm{Mdd} \le M_{\mathrm{var}} \end{cases}$$

Transonic pitching-moment coefficient:

```
Cm_trans
```

```
Cm_trans =
```

$$\begin{cases} \mathrm{Cm}_{0,\mathrm{ac}} + \mathrm{Cma}_{\mathrm{ac}}\,\alpha + \mathrm{Cmde}\,\mathrm{de} + h_{\mathrm{cg}} \left(\mathrm{CL}_0 + \mathrm{CLa}\,\alpha + \mathrm{CLde}\,\mathrm{de} + \dfrac{\mathrm{CLq}\,\mathrm{cbar}\,q}{2\,V}\right) + \dfrac{\mathrm{Cmq}\,\mathrm{cbar}}{2\,V} \\ \mathrm{Cm}_{0,\mathrm{ac}} + \mathrm{Cma}_{\mathrm{ac}}\,\alpha + \mathrm{Cmde}\,\mathrm{de} + \mathrm{CmMach}\ (M_{\mathrm{var}} - \mathrm{Mdd})^2 + h_{\mathrm{cg}} \left(\mathrm{CL}_0 + \mathrm{CLa}\,\alpha + \mathrm{CLde}\,\mathrm{de} + \dfrac{\mathrm{CLq}\,\mathrm{cbar}\,q}{2\,V}\right) \end{cases}$$

Rebuild and Re-Trim the Transonic Model

Once pitching moment depends on Mach, the elevator trim changes. Re-trimming before linearization keeps the A and B matrices tied to a true operating point.

```
D_trans = qbar*S*CD_transonic;
M_moment_trans = qbar*S*cbar*Cm_transonic;
Fx_wind_trans = thrust*cos(alpha) - D_trans - mass*g*sin(gamma);
Fz_wind_trans = -thrust*sin(alpha) - L_force + mass*g*cos(gamma);
f_trans = [Fx_wind_trans/mass; ...
    Fz_wind_trans/(mass*V) + q; ...
    M_moment_trans/Iyy; ...
```

```
    q];
Mdd_val = 0.72;
Kwave_val = 0.12;
CmMach_val = -0.20;
params_trans = [params, a_sym, Mdd, Kwave, CmMach];
vals_trans = [vals, a_sound, Mdd_val, Kwave_val, CmMach_val];
thrustTrimExpr_trans = (D_trans + mass*g*sin(gamma))/cos(alpha);
trim_eqs_trans = subs([subs(Fz_wind_trans, thrust, thrustTrimExpr_trans) == 0; M_moment_trans == 0], ...
    [params_trans, V, q, theta], ...
    [vals_trans, V0, 0, alpha]);
trim_sol_trans = vpasolve(trim_eqs_trans, [alpha, de], [alpha0, de0]);
alpha0_trans = double(trim_sol_trans.alpha);
de0_trans = double(trim_sol_trans.de);
theta0_trans = alpha0_trans;
thrust0_trans = double(subs(thrustTrimExpr_trans, ...
    [params_trans, V, alpha, q, theta, de], ...
    [vals_trans, V0, alpha0_trans, 0, theta0_trans, de0_trans]));
trimResidual_trans = double(subs(f_trans, [params_trans, states, controls], ...
    [vals_trans, V0, alpha0_trans, 0, theta0_trans, de0_trans, thrust0_trans]));
disp(join([
    compose("Transonic trim AoA: %.2f deg", rad2deg(alpha0_trans))
    compose("Transonic trim elevator: %.2f deg", rad2deg(de0_trans))
    compose("Transonic trim thrust: %.0f N", thrust0_trans)
    compose("Transonic trim residual norm: %.2e", norm(trimResidual_trans))
], newline))
```

```
Transonic trim AoA: 3.90 deg
Transonic trim elevator: -2.06 deg
Transonic trim thrust: 35136 N
Transonic trim residual norm: 2.93e-17
```

Transonic A/B Matrices and Derivative Comparison

The wave-drag term mainly changes X_u. The Mach-dependent moment term creates the nonzero M_u expected from Mach tuck.

```
A_trans_sym = simplify(jacobian(f_trans, states));
B_trans_sym = simplify(jacobian(f_trans, controls));
A_trans_Val = double(subs(A_trans_sym, [params_trans, states, controls], ...
```

```
    [vals_trans, V0, alpha0_trans, 0, theta0_trans, de0_trans, thrust0_trans]));
B_trans_Val = double(subs(B_trans_sym, [params_trans, states, controls], ...
    [vals_trans, V0, alpha0_trans, 0, theta0_trans, de0_trans, thrust0_trans]));
Xu_trans = A_trans_Val(1, 1);
Mu_trans = A_trans_Val(3, 1);
dXu_dKwave = diff(A_trans_sym(1, 1), Kwave);
dMu_dCmMach = diff(A_trans_sym(3, 1), CmMach);
dXu_dKwave_Val = double(subs(dXu_dKwave, [params_trans, states, controls], ...
    [vals_trans, V0, alpha0_trans, 0, theta0_trans, de0_trans, thrust0_trans]));
dMu_dCmMach_Val = double(subs(dMu_dCmMach, [params_trans, states, controls], ...
    [vals_trans, V0, alpha0_trans, 0, theta0_trans, de0_trans, thrust0_trans]));
fprintf(['=== Subsonic vs Transonic Derivatives ===\n' ...
    'X_u: subsonic = %+.6f, transonic = %+.6f /s\n' ...
    'M_u: subsonic = %+.6f, transonic = %+.6f /s^2\n' ...
    'dX_u/dKwave = %+.6f /s per unit\n' ...
    'dM_u/dCmMach = %+.6f /s^2 per unit\n'], ...
    Xu_Val, Xu_trans, Mu_Val, Mu_trans, dXu_dKwave_Val, dMu_dCmMach_Val)
```

```
=== Subsonic vs Transonic Derivatives ===
X_u: subsonic = -0.005047, transonic = -0.005144 /s
M_u: subsonic = +0.000000, transonic = -0.000123 /s^2
dX_u/dKwave = -0.000810 /s per unit
dM_u/dCmMach = +0.000664 /s^2 per unit
```

Updated Modal Analysis

```
eigvals_trans = eig(A_trans_Val);
wn_trans = abs(eigvals_trans);
zeta_trans = -real(eigvals_trans)./wn_trans;
periods_trans = 2*pi./abs(imag(eigvals_trans));
[~, idx_t] = sort(wn_trans, "descend");
fprintf(['=== Modal Comparison ===\n' ...
    '                    Subsonic         Transonic\n' ...
    'SP wn (rad/s):      %.3f           %.3f\n' ...
    'SP zeta:            %.3f           %.3f\n' ...
    'Phugoid wn:         %.4f          %.4f\n' ...
    'Phugoid zeta:       %.4f          %.4f\n' ...
    'Phugoid period (s): %.0f             %.0f\n'], ...
```

```
    wn(sp_idx(1)), wn_trans(idx_t(1)), ...
    zeta(sp_idx(1)), zeta_trans(idx_t(1)), ...
    wn(ph_idx(1)), wn_trans(idx_t(3)), ...
    zeta(ph_idx(1)), zeta_trans(idx_t(3)), ...
    periods(ph_idx(1)), periods_trans(idx_t(3)))
```

```
=== Modal Comparison ===
                    Subsonic         Transonic
SP wn (rad/s):      1.355            1.355
SP zeta:            0.284            0.284
Phugoid wn:         0.0578           0.0548
Phugoid zeta:       0.0339           0.0349
Phugoid period (s): 109              115
```

5. Code Generation for Downstream Use

Generate one function for nonlinear simulation and one function for the local state-space model. These generated files are the bridge from symbolic derivation to engineering workflows such as control design, envelope sweeps, and Simulink integration.

```
modelVars = {V, alpha, q, theta, de, thrust, ...
    CL0, CLa, CLq, CLde, CD0, K_drag, ...
    Cm0_ac, Cma_ac, h_cg, Cmq, Cmde, ...
    S, cbar, mass, Iyy, rho, g, a_sym, Mdd, Kwave, CmMach};
modelVals = {V0, alpha0_trans, 0, theta0_trans, de0_trans, thrust0_trans, ...
    CL0_val, CLa_val, CLq_val, CLde_val, CD0_val, K_val, ...
    Cm0ac_val, Cmaac_val, h_cg_val, Cmq_val, Cmde_val, ...
    S_val, cbar_val, mass_val, Iyy_val, rho_val, g_val, a_sound, Mdd_val, Kwave_val, CmMach_val};
matlabFunction(f_trans, 'File', 'longitudinalEOM', ...
    'Vars', modelVars, ...
    'Outputs', {'xdot'}, ...
    'Optimize', true);
matlabFunction(A_trans_sym, B_trans_sym, 'File', 'longitudinalStateSpace', ...
    'Vars', modelVars, ...
    'Outputs', {'A', 'B'}, ...
    'Optimize', true);
disp("Generated: longitudinalEOM.m")
```

```
Generated: longitudinalEOM.m
```

```
disp("Generated: longitudinalStateSpace.m")
```

```
Generated: longitudinalStateSpace.m
```

Verify Generated Code

Check that generated numeric functions reproduce the symbolic evaluation at the transonic trim point.

```
F_generated = longitudinalEOM(modelVals{:});
[A_generated, B_generated] = longitudinalStateSpace(modelVals{:});
F_symbolic_Val = double(subs(f_trans, [params_trans, states, controls], ...
    [vals_trans, V0, alpha0_trans, 0, theta0_trans, de0_trans, thrust0_trans]));
generatedResidual = norm(F_generated - F_symbolic_Val);
stateSpaceResidual = norm(A_generated - A_trans_Val, "fro") + norm(B_generated - B_trans_Val, "fro");
fprintf(['=== Generated Function Check ===\n' ...
    'EOM residual: %.3e\n' ...
    'A/B residual: %.3e\n'], generatedResidual, stateSpaceResidual)
```

```
=== Generated Function Check ===
EOM residual: 3.545e-17
A/B residual: 2.785e-16
```

6. Stability-Derivative Uncertainty

Wind-tunnel and flight-test data carry uncertainty. Here the generated state-space function evaluates hundreds of local models while keeping the operating point fixed at the nominal transonic trim.

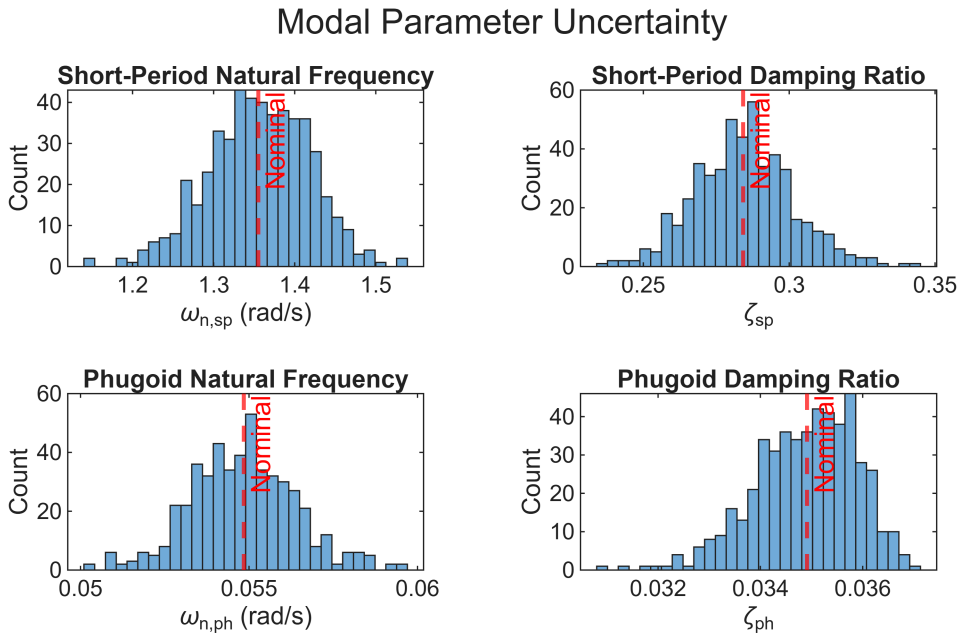
```
sigma_CLa = 0.55;
sigma_Cmaac = 0.045;
sigma_hcg = 0.01;
sigma_Cmq = 2.7;
N_mc = 500;
rng(17)
CLa_samples = CLa_val + sigma_CLa*randn(N_mc, 1);
Cmaac_samples = Cmaac_val + sigma_Cmaac*randn(N_mc, 1);
hcg_samples = h_cg_val + sigma_hcg*randn(N_mc, 1);
Cmq_samples = Cmq_val + sigma_Cmq*randn(N_mc, 1);
wn_sp_mc = zeros(N_mc, 1);
```

```
zeta_sp_mc = zeros(N_mc, 1);
wn_ph_mc = zeros(N_mc, 1);
zeta_ph_mc = zeros(N_mc, 1);
Malpha_mc = zeros(N_mc, 1);
for k = 1:N_mc
    modelVals_k = modelVals;
    modelVals_k{8} = CLa_samples(k);
    modelVals_k{14} = Cmaac_samples(k);
    modelVals_k{15} = hcg_samples(k);
    modelVals_k{16} = Cmq_samples(k);
    [A_k, ~] = longitudinalStateSpace(modelVals_k{:});
    eigk = eig(A_k);
    wnk = abs(eigk);
    zetak = -real(eigk)./wnk;
    [~, sortk] = sort(wnk, "descend");
    wn_sp_mc(k) = wnk(sortk(1));
    zeta_sp_mc(k) = zetak(sortk(1));
    wn_ph_mc(k) = wnk(sortk(3));
    zeta_ph_mc(k) = zetak(sortk(3));
    Malpha_mc(k) = A_k(3, 2);
end
```

Modal Uncertainty Distribution

```
tiledlayout(2, 2)
nexttile
histogram(wn_sp_mc, 30)
xline(wn_trans(idx_t(1)), "r--", "Nominal", "LineWidth", 1.5)
xlabel("\omega_{n,sp} (rad/s)")
ylabel("Count")
title("Short-Period Natural Frequency")
nexttile
histogram(zeta_sp_mc, 30)
xline(zeta_trans(idx_t(1)), "r--", "Nominal", "LineWidth", 1.5)
xlabel("\zeta_{sp}")
ylabel("Count")
title("Short-Period Damping Ratio")
```

```
nexttile
histogram(wn_ph_mc, 30)
xline(wn_trans(idx_t(3)), "r--", "Nominal", "LineWidth", 1.5)
xlabel("\omega_{n,ph} (rad/s)")
ylabel("Count")
title("Phugoid Natural Frequency")
nexttile
histogram(zeta_ph_mc, 30)
xline(zeta_trans(idx_t(3)), "r--", "Nominal", "LineWidth", 1.5)
xlabel("\zeta_{ph}")
ylabel("Count")
title("Phugoid Damping Ratio")
sgtitle("Modal Parameter Uncertainty")
```



Confidence Bounds and Sensitivity Check

Compute percentile bounds directly from sorted samples, then verify the exact symbolic $\partial M_\alpha/\partial C_{L\alpha}$ against a one-at-a-time generated-function sweep.

```
lo_idx = max(1, ceil(0.025*N_mc));
hi_idx = min(N_mc, floor(0.975*N_mc));
wn_sp_sorted = sort(wn_sp_mc);
zeta_sp_sorted = sort(zeta_sp_mc);
wn_ph_sorted = sort(wn_ph_mc);
zeta_ph_sorted = sort(zeta_ph_mc);
fprintf(['=== 95%% Sample Bounds ===\n' ...
    'SP wn:   [%.3f, %.3f] rad/s  (nominal: %.3f)\n' ...
    'SP zeta: [%.3f, %.3f]        (nominal: %.3f)\n' ...
    'Ph wn:   [%.4f, %.4f] rad/s (nominal: %.4f)\n' ...
    'Ph zeta: [%.4f, %.4f]       (nominal: %.4f)\n'], ...
    wn_sp_sorted(lo_idx), wn_sp_sorted(hi_idx), wn_trans(idx_t(1)), ...
    zeta_sp_sorted(lo_idx), zeta_sp_sorted(hi_idx), zeta_trans(idx_t(1)), ...
    wn_ph_sorted(lo_idx), wn_ph_sorted(hi_idx), wn_trans(idx_t(3)), ...
    zeta_ph_sorted(lo_idx), zeta_ph_sorted(hi_idx), zeta_trans(idx_t(3)))
```

```
=== 95% Sample Bounds ===
SP wn:   [1.230, 1.469] rad/s  (nominal: 1.355)
SP zeta: [0.252, 0.319]        (nominal: 0.284)
Ph wn:   [0.0517, 0.0582] rad/s (nominal: 0.0548)
Ph zeta: [0.0327, 0.0365]       (nominal: 0.0349)
```

```
dMalpha_dCLa_sym = diff(A_trans_sym(3, 2), CLa);
dMalpha_dCLa_Val = double(subs(dMalpha_dCLa_sym, [params_trans, states, controls], ...
    [vals_trans, V0, alpha0_trans, 0, theta0_trans, de0_trans, thrust0_trans]));
CLa_sweep = CLa_val + linspace(-1, 1, 21)*sigma_CLa;
Malpha_sweep = zeros(size(CLa_sweep));
for k = 1:numel(CLa_sweep)
    modelVals_k = modelVals;
    modelVals_k{8} = CLa_sweep(k);
    [A_k, ~] = longitudinalStateSpace(modelVals_k{:});
    Malpha_sweep(k) = A_k(3, 2);
end
p_malpha = polyfit(CLa_sweep, Malpha_sweep, 1);
fprintf(['\nSensitivity d(M_alpha)/d(CLa):\n' ...
    '  Symbolic derivative: %.4f /s^2 per unit\n' ...
    '  Generated sweep:     %.4f /s^2 per unit\n' ...
    '  Difference:          %.3e\n'], ...
```

```
    dMalpha_dCLa_Val, p_malpha(1), abs(dMalpha_dCLa_Val - p_malpha(1)))
```

```
Sensitivity d(M_alpha)/d(CLa):
  Symbolic derivative: -0.2285 /s^2 per unit
  Generated sweep:     -0.2285 /s^2 per unit
  Difference:          3.053e-16
```